RANDOM POSITION EXPERIMENT
STEPPER MOTOR RPM CONTROL

Closed-Loop PID Speed Control using Raspberry Pi 4

Project	Random Position Experiment — Stepper Motor RPM Control
Motor	PR42H34-1334BED (NEMA 17, 1.33A)
Driver	TB6600 Microstep Driver
Controller	Raspberry Pi 4 Model B
Supply Voltage	12 V DC
Microstep Used	3200 pulse/rev (S1=OFF S2=OFF S3=ON)
Stable RPM Range	3 – 50 RPM (3200 microsteps)
Tuning Method	Ziegler-Nichols (Classic and Moderate modes)
Future Plan	Two-motor combined circuit for random position control

Vijay Kumar | 2026

1. Components and Hardware

1.1 Bill of Materials

Component	Specification	Notes
Stepper Motor	PR42H34-1334BED	NEMA 17, 1.33A rated, 500 PPR built-in encoder
Motor Driver	TB6600	9–42 V input, up to 4 A, DIP switch configuration
Microcontroller	Raspberry Pi 4 Model B	Quad-core, 3.3 V GPIO logic
Power Supply	12 V DC	Connected to TB6600 VCC+
Encoder	Built-in 500 PPR quadrature	2000 counts/rev measured (CHANGE mode)
Current tested	1.33 A and 1.5 A	Both settings tested on TB6600

1.2 Motor Coil Wire Colour Code

Wire Colour	Signal	TB6600 Terminal	Notes
Black	A+	A+	Coil A positive
White	A−	A−	Coil A negative

Red	B+	B+	Coil B positive
Yellow	B−	B−	Coil B negative

1.3 TB6600 DIP Switch Settings — 3200 Microsteps

The final selected microstep setting is 3200 pulses/rev (16× microstep). This gives the best balance between smoothness and RPi4 pulse generation capability for the 3–50 RPM operating range.

Switches	S1	S2	S3	S4	S5	S6
3200 Microstpes	OFF	OFF	ON	ON	ON	OFF
6400 Microsteps	OFF	OFF	OFF	ON	ON	OFF



Figure 1 — TB6600 driver with DIP switch reference table (S1–S3 microstep, S4–S6 current)

2. Wiring and GPIO Signal Connections

2.1 GPIO Pin Mapping

Signal	RPi GPIO (BCM)	TB6600 / Encoder	Description
STEP	GPIO 17	PUL+	Step pulse — one pulse advances motor one microstep
DIR	GPIO 27	DIR+	Direction — HIGH = clockwise rotation
ENA	GPIO 22	ENA+	Enable — LOW = active, HIGH = driver disabled (coils off)
Encoder A+	GPIO 24	Orange wire	Quadrature channel A — hardware interrupt on BOTH edges
Encoder B+	GPIO 23	Blue wire	Quadrature channel B — read at each A edge

Encoder VCC	3.3 V (PIN 1)	Red wire	⚠ Must be 3.3 V — NOT 5 V
Encoder GND	GND	Black wire	Common ground
Common GND	GND	PUL− DIR− ENA−	All grounds tied together with PSU GND
12 V power	—	VCC+	12 V PSU → TB6600 motor supply

2.2 Signal Flow Diagram

```
  User / Random-mode sets target_rpm
          |
          v
  [ PID Controller — 100 ms loop ]  <===  Encoder feedback
    error = target_rpm - filtered_rpm       GPIO24 (A+)
    command_rpm = target + Kp*err + Ki*integral + Kd*deriv   GPIO23 (B+)
          |
          v
  [ Step Thread ]
    freq = command_rpm x MICROSTEPS / 60
    half_period = 1 / (2 x freq)
          |
          v
  GPIO17 (PUL+) ----> TB6600 ----> Motor coils A / B
  GPIO27 (DIR+) ----> TB6600       (Black/White = A, Red/Yellow = B)
  GPIO22 (ENA+) ----> TB6600
                        |
                  Shaft rotates
                        |
              Encoder (A+ orange, B+ blue) <--+
```

3. Encoder and RPM Calculation

3.1 Quadrature Counting

The 500 PPR encoder produces channels A and B. The software polls GPIO24 (A+) every 50 µs and detects state changes. Channel B (GPIO23) is read at the same moment to determine direction. This gives 1000 edges per revolution. After calibration, 2000 counts/rev was measured.

3.2 RPM Formula (from GUI code — gui_code.py, line 151)

```
  delta_counts  =  enc_count_now  -  enc_count_100ms_ago

  actual_RPM  =  ( |delta_counts| / COUNTS_PER_REV )  x  600

  where  COUNTS_PER_REV = 2000  (calibrated value)
         600  =  60 sec/min  x  10  (because sample period = 100 ms = 0.1
s)
              =  60 / 0.1

  Python:  actual_rpm = (abs(delta) / COUNTS_PER_REV) * 600.0
```

3.3 Low-Pass Filter

```
  filtered_rpm = (1 - FILTER_ALPHA) x filtered_rpm  +  FILTER_ALPHA x
actual_rpm

  FILTER_ALPHA = 0.3  (from gui_code.py)
  Lower alpha = smoother but slower.  Higher = faster but noisier.
```

4. Microstep Selection — Why 3200

4.1 Pulse Frequency Constraint

The Raspberry Pi 4 runs Linux which introduces timing jitter of ±1–5 ms in Python time.sleep(). The required pulse frequency for a given RPM is:

```
  Required freq (Hz)  =  RPM  x  MICROSTEPS  /  60

  At 50 RPM with 3200 microsteps:  freq  =  50 x 3200 / 60  =  2667 Hz
  At 36 RPM with 6400 microsteps:  freq  =  36 x 6400 / 60  =  3840 Hz
```

4.2 Tested Microstep Settings

Microstep	S1 S2 S3	Max stable RPM	Smoothness	Decision
3200	OFF OFF ON	50 RPM	Very smooth	✓ Selected
6400	OFF OFF OFF	36 RPM	Smoother	Not selected

```
  3200 microsteps chosen because:
  1. Handles RPM range 3–50 required by the Random Position Experiment
  2. Sufficient smoothness for the application
  3. Pulse frequency at 50 RPM = 2667 Hz — within RPi4 software capability
  6400 microsteps rejected: max stable RPM = 36, insufficient for the
experiment
```

5. Transfer Function and System Model

5.1 First Order Plus Dead Time (FOPDT) Model

The motor + driver + software system is modelled as a First Order Plus Dead Time (FOPDT) process. This is identified from the step response logged data.

```
  G(s)  =  K  x  e^(-Ls)  /  (Ts + 1)

  K  =  0.459   (process gain — RPM output per RPM commanded)
  T  =  0.41 s  (time constant — time to reach 63.2% of final RPM)
  L  =  0.10 s  (dead time — one PID sample period delay)

  G(s)  =  0.459  x  e^(-0.10s)  /  (0.41s + 1)
```

5.2 Simulated Closed-Loop Step Response

Using the identified transfer function, the following graphs show simulated closed-loop performance for Classic and Moderate ZN gains at 30 RPM and 50 RPM targets.

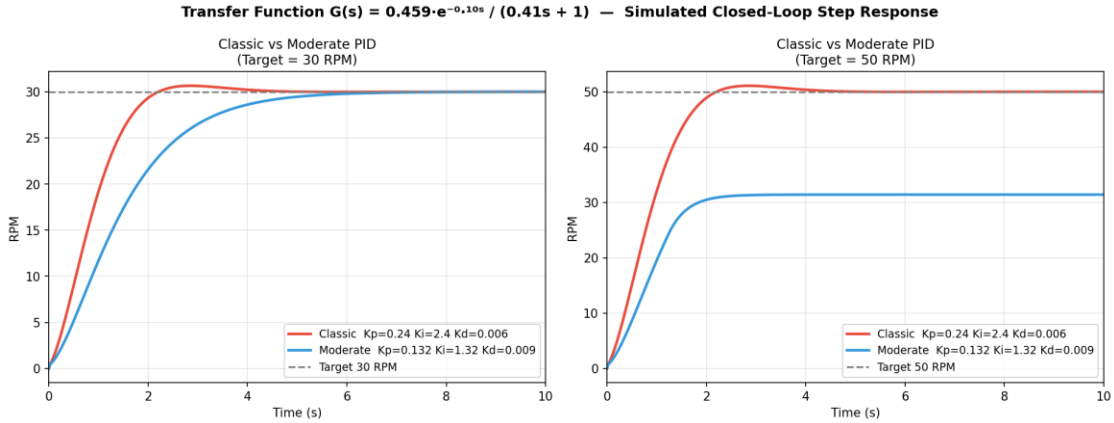


Figure 2 — Simulated closed-loop step response: Classic vs Moderate PID at 30 RPM (left) and 50 RPM (right)

6. PID Tuning — Ziegler-Nichols Method

6.1 Procedure

6.2 ZN Formulas and Calculated Gains

```
── Classic PID ──────────────────────────────────────────────────────
Kp  =  0.6  x  Ku          =  0.6  x  0.40          =  0.240
Ki  =  2  x  Kp  /  Tu     =  2  x  0.240  /  0.20  =  2.400
Kd  =  Kp  x  Tu  /  8     =  0.240  x  0.20  /  8  =  0.006

── Moderate PID ─────────────────────────────────────────────────────
Kp  =  0.33  x  Ku         =  0.33  x  0.40         =  0.132
Ki  =  2  x  Kp  /  Tu     =  2  x  0.132  /  0.20  =  1.320
Kd  =  Kp  x  Tu  /  3     =  0.132  x  0.20  /  3  =  0.009
```

Mode	Kp	Ki	Kd	Characteristic
Classic	0.240	2.400	0.006	Faster response, small overshoot
Moderate	0.132	1.320	0.009	Smoother, less overshoot

6.3 Additional Features

Deadband Lock

When |error| < 0.8 RPM for 3 consecutive PID cycles (0.3 s), command_rpm is frozen and the integrator is held. This prevents hunting at steady state.

```
if abs(error) <= 0.8:
    deadband_counter += 1
    if deadband_counter >= 3:  command_rpm = locked_command
```

Ramp Limiter

```
RAMP_UP   = 1.0  RPM per step loop cycle
RAMP_DOWN = 1.5  RPM per step loop cycle
Prevents sudden step changes to the motor.
```

7. Experimental Results

All results use MICROSTEPS = 3200, 12 V supply, COUNTS_PER_REV = 2000, FILTER_ALPHA = 0.3, DEADBAND = ±0.8 RPM.

7.1 Classic PID — Sweep Mode (5 to 50 RPM)

7.2 Moderate PID — Sweep Mode

Moderate ZN gains: Kp=0.132, Ki=1.32, Kd=0.009. Smoother response, slightly slower to reach target.

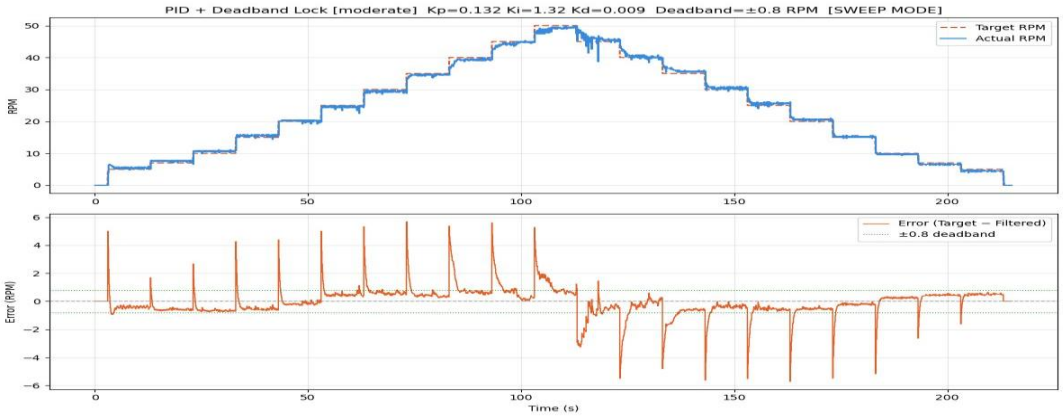


Figure 5 — Moderate PID, Sweep mode: RPM tracking

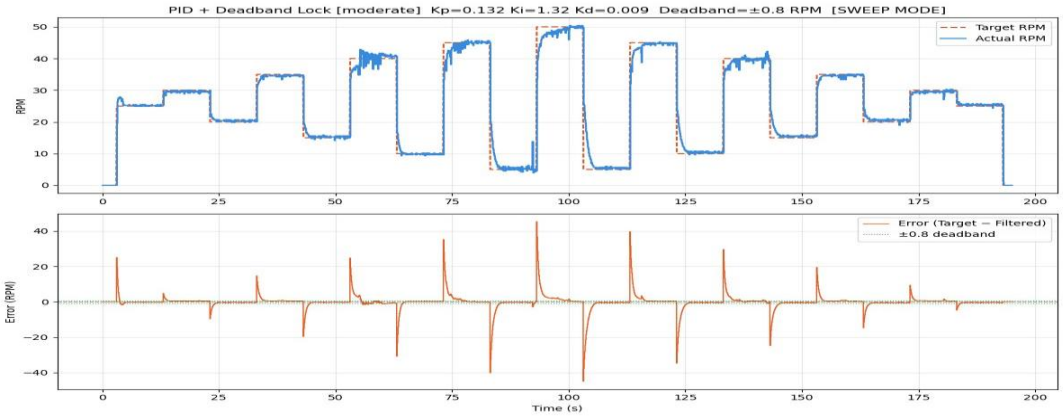


Figure 6 — Moderate PID, Sweep mode: second run

7.3 Classic vs Moderate — Side-by-Side Comparison (Sweep)

Direct comparison of Classic and Moderate gains under identical sweep conditions. Both graphs show the same RPM profile.

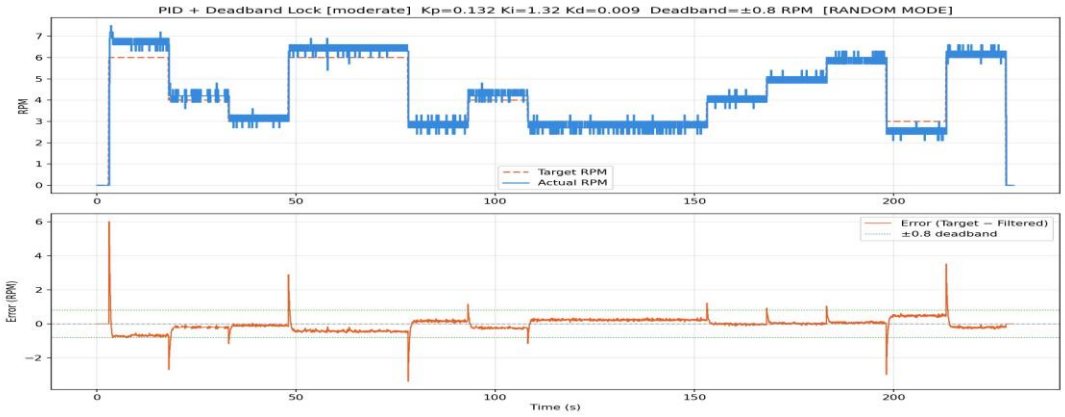


Figure 7 — Classic PID full sweep session (Kp=0.24, Ki=2.4, Kd=0.006)

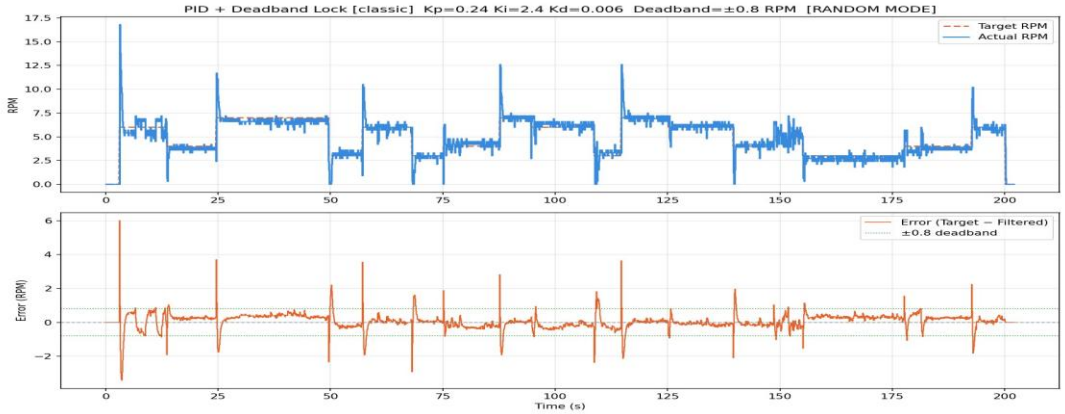


Figure 8 — Moderate PID full sweep session (Kp=0.132, Ki=1.32, Kd=0.009)

7.4 Random Mode — Classic PID

Random mode assigns a new random RPM from a defined range at random time intervals. This simulates the unpredictable nature of the Random Position Experiment.

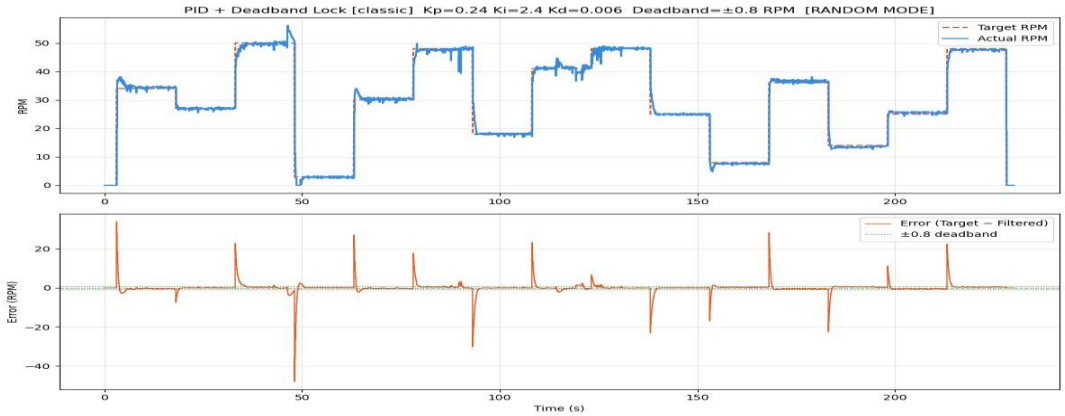


Figure 9 — Classic PID, Random mode: motor tracks unpredictable RPM targets

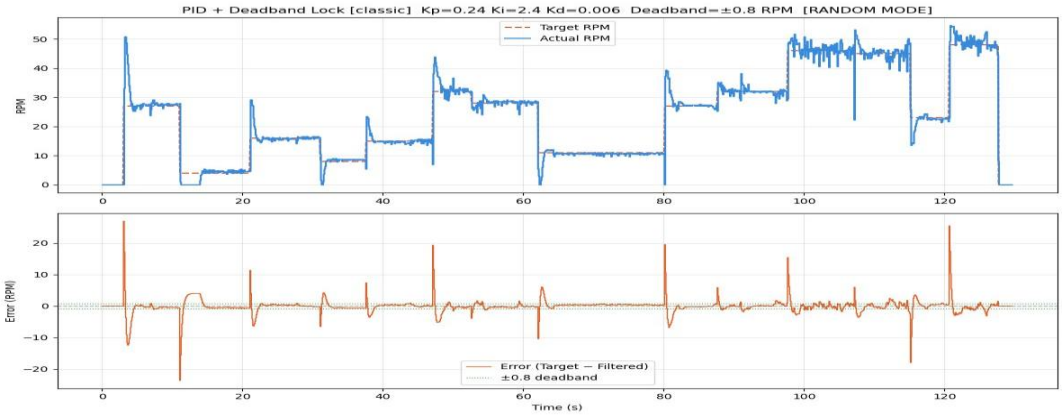


Figure 10 — Classic PID, full random session: all RPM levels including high range 25–50 RPM

7.5 Random Mode — Moderate PID

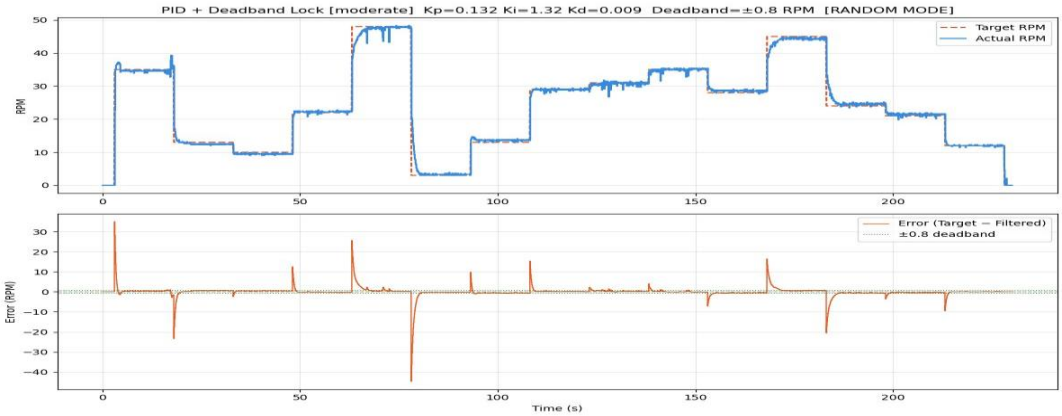


Figure 11 — Moderate PID, Random mode

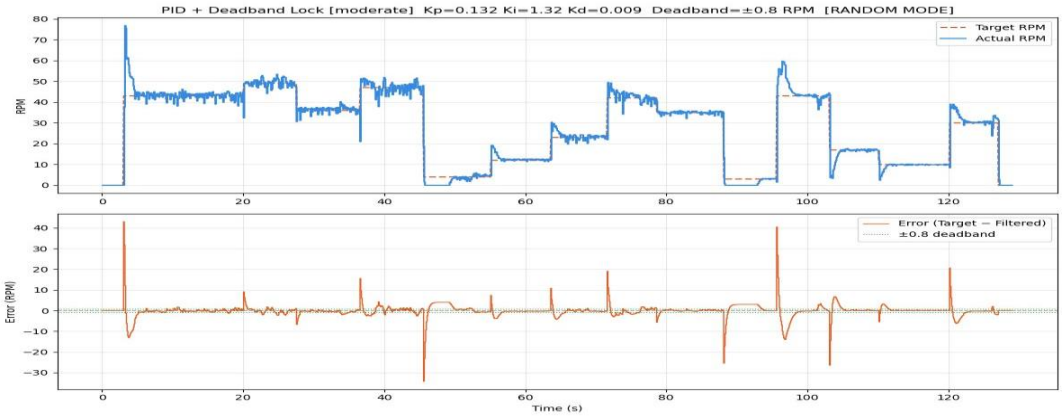


Figure 12 — Moderate PID, full random session

7.6 Optimized Random Mode

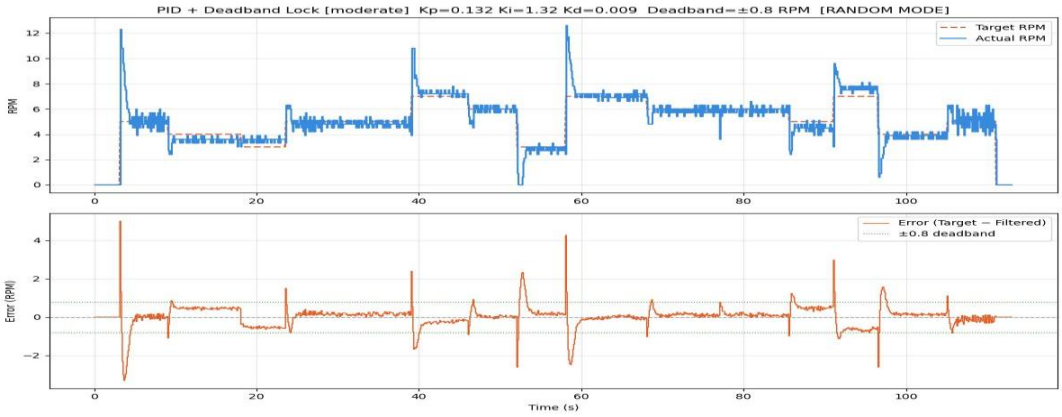


Figure 13 — Optimized random mode: best overall tracking performance

7.7 Low RPM Range Test (3 – 7 RPM)

Testing at very low RPM (3–7 RPM) verifies the system works at the low end of the Random Position Experiment range. Both tuning modes compared.

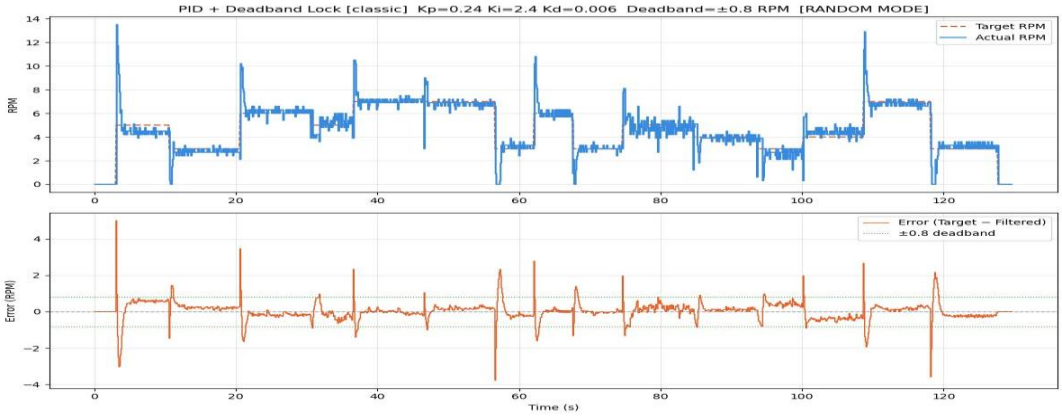


Figure 14 — Classic PID, 3–7 RPM random mode: tracks well with ±0.8 RPM deadband

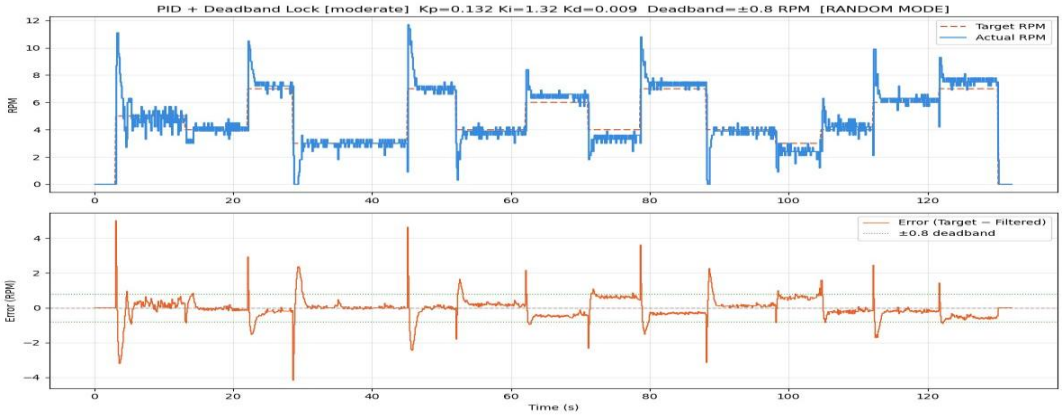


Figure 15 — Moderate PID, 3–7 RPM random mode

7.8 Performance Summary

Test	Mode	RPM range	Steady err	Transient	Result
Sweep 5–50 RPM	Classic	5–50	±1–2	±5 on step	✓ Excellent
Sweep 5–50 RPM	Moderate	5–50	±1–2	±3 on step	✓ Excellent
Random 4–48 RPM	Classic	4–48	±1–3	±25 on step	✓ Good
Random 10–45 RPM	Moderate	10–45	±1–2	±40 on step	✓ Good
Low RPM 3–7	Both	3–7	±0.5	±5 on step	✓ Best

8. How to Run the Code

8.1 Installation

5. Install lgpio: sudo apt install python3-lgpio python3-matplotlib python3-tkinter
6. Copy gui_code.py and stepper_zn_pid_random.py to the Raspberry Pi

8.2 Running from Terminal (command-line)

```
cd /home/Ashutosh/Downloads
sudo python3 optimized .py
```

At the prompt, select run mode by editing RUN_MODE in the script header:

```
RUN_MODE = "sweep"   → Auto steps through SWEEP_RPMS list
RUN_MODE = "random"  → Random RPM, random dwell time
RUN_MODE = "manual"  → Type RPM at prompt
```

8.3 Running with GUI

```
cd /home/Ashutosh/Downloads
sudo python3 gui_code.py
```

The GUI provides:

- ZN mode selector (classic / moderate / no_overshoot)
- Run mode selector (sweep / manual / random)
- Random mode config: min/max RPM, min/max dwell time, number of steps
- Sweep RPM list and dwell time config
- Live terminal log with colour-coded status
- Automatic graph saved as PNG when session ends

9. Drawbacks and Limitations

9.1 Linux OS Timing Jitter (Primary Limitation)

The most significant limitation is that Python's time.sleep() on a Linux operating system cannot guarantee precise timing. The Linux scheduler introduces jitter of ±1–5 ms.

```
At 50 RPM with 3200 microsteps:
  Required half-period  =  1 / (2 x 2667)  =  0.188 ms
  Python jitter          =  ±1-5 ms
  Ratio                  =  5-27× larger than required period

Effect: Step pulses arrive irregularly → motor vibrates slightly
        at higher RPM. PID compensates but cannot fully eliminate
        the underlying timing noise.

Solution: Use lgpio.tx_pwm() (hardware PWM) for pulse generation.
          This runs in kernel space with µs-level precision.
```

Limitation	Impact and Mitigation
Linux time.sleep() jitter ±1–5ms	Irregular step pulses at high RPM. Mitigated by using 3200 microsteps (lower freq required).
Encoder polling (50µs poll)	May miss edges at very high RPM. Below 50 RPM all edges captured reliably.
12 V supply (not 24 V)	Lower torque at high RPM. Adequate for 3–50 RPM range.
Open-loop driver (TB6600)	Cannot self-correct missed steps. Software PID compensates via encoder feedback.
Single motor only	Future work: two-motor combined circuit for full random position experiment.

10. Future Work — Two-Motor Combined System

The current implementation controls a single NEMA 17 stepper motor. The full Random Position Experiment requires two motors operating simultaneously with independent closed-loop PID control.

10.1 Planned Two-Motor Circuit

Item	Specification
Motor 1	PR42H34-1334BED — same as current
Motor 2	PR42H34-1334BED — identical second unit
Driver 1	TB6600 — STEP GPIO17, DIR GPIO27, ENA GPIO22
Driver 2	TB6600 — STEP GPIO5, DIR GPIO6, ENA GPIO13
Encoder 1 A+/B+	GPIO24 / GPIO23
Encoder 2 A+/B+	GPIO25 / GPIO8

Power supply	Two TB6600 units, shared 12 V PSU (minimum 5 A)

10.2 Code Logic for Two Motors

Each motor will have its own independent set of threads and PID state:

```
Motor 1 threads:  encoder_thread_1, pid_thread_1, step_thread_1
Motor 2 threads:  encoder_thread_2, pid_thread_2, step_thread_2

Shared coordinator thread:
  - Generates random RPM pairs (rpm1, rpm2)
  - Sends each to respective target_rpm_1 and target_rpm_2
  - Waits for both to lock (deadband achieved)
  - Records position, logs data, generates new random pair

Application: motor 1 controls angular position shaft A
             motor 2 controls angular position shaft B
             Together they define a 2D random position in the experiment
```

11. Conclusion and Recommended Settings

Parameter	Final Setting
Microstep (S1 S2 S3)	3200 → OFF OFF ON
Current (S4 S5 S6)	1.5 A → ON ON OFF
Supply voltage	12 V DC
COUNTS_PER_REV	2000 (calibrated)
ZN tuning mode	Classic (Kp=0.24 Ki=2.4 Kd=0.006)
FILTER_ALPHA	0.3
DEADBAND_RPM	0.8 RPM (lock after 3 cycles = 0.3 s)
Stable RPM range	3 – 50 RPM
Application	Random Position Experiment — single motor (dual motor in future)

The closed-loop PID system with 3200 microstep setting successfully controls the NEMA 17 stepper motor across the 3–50 RPM range required by the Random Position Experiment. Classic ZN gains provide the best step-response tracking. The deadband lock eliminates steady-state hunting. The primary remaining limitation is Linux OS timing jitter which can be resolved by switching to lgpio hardware PWM for pulse generation in future work.